

Docker

A Comprehensive Guide to Containerization

Topics Covered	Chapters
Containers & Images	10 Chapters
Dockerfile Mastery	50+ Pages
Networking & Volumes	Code Examples
Docker Compose	Best Practices
Security & Production	Real-world Use Cases

Table of Contents

1. Introduction to Docker	3
2. Docker Architecture & Core Concepts	6
3. Installing Docker	10
4. Working with Images	13
5. Dockerfile: Writing & Optimizing	17
6. Container Lifecycle Management	22
7. Docker Networking	26
8. Volumes & Persistent Storage	31
9. Docker Compose	35
10. Docker Security & Best Practices	43
11. Docker in Production	47

Chapter 1: Introduction to Docker

Docker is an open-source platform that automates the deployment, scaling, and management of applications inside lightweight, portable containers. Since its public release in 2013 by Solomon Hykes at dotCloud (later renamed Docker, Inc.), it has fundamentally transformed how software is built, shipped, and run across different environments.

Before Docker, the classic developer complaint was 'it works on my machine.' Different environments — development laptops, staging servers, and production clusters — had subtly different library versions, OS patches, and configuration, leading to endless debugging and deployment failures. Docker solves this by packaging an application and everything it needs (runtime, libraries, config) into a single, self-contained unit called a container.

1.1 The Problem Docker Solves

Traditional software deployment faced several critical challenges that Docker was designed to address:

- **Environment Inconsistency:** Code behaved differently across dev, staging, and production due to varying OS versions, library conflicts, and configuration drift.
- **Dependency Hell:** Installing multiple applications on the same server caused library version conflicts that were difficult to resolve.
- **Slow Provisioning:** Spinning up new servers or virtual machines took hours or days, making rapid scaling impractical.
- **Resource Inefficiency:** Virtual machines (VMs) consume large amounts of RAM and CPU because each one runs a full OS kernel.
- **Poor Portability:** Applications built for one environment were hard to move to another without significant rework.

1.2 What is a Container?

A container is a standardized unit of software that packages code and all its dependencies so the application runs quickly and reliably from one computing environment to another. Containers isolate software from its environment and ensure it works uniformly despite differences between development and staging environments.

Unlike virtual machines, containers do not bundle a full OS. Instead, they share the host system's OS kernel and isolate the application processes from each other using Linux kernel features like namespaces and cgroups. This makes containers much lighter and faster than VMs.

1.3 Containers vs. Virtual Machines

Feature	Containers	Virtual Machines
---------	------------	------------------

Boot Time	Milliseconds	Minutes
Size	Megabytes	Gigabytes
OS	Shares host kernel	Full OS per VM
Isolation	Process-level	Hardware-level
Portability	Very High	Moderate
Performance	Near-native	Overhead present
Density	Hundreds per host	Dozens per host

1.4 Key Benefits of Docker

Docker has become the industry standard for containerization for a number of compelling reasons:

- **Consistency:** 'Build once, run anywhere.' The same Docker image runs identically on a developer's laptop, a CI/CD server, and a production Kubernetes cluster.
- **Speed:** Containers start in milliseconds. You can scale a service up or down almost instantly.
- **Efficiency:** Many containers share the host OS kernel, using far less RAM and disk than equivalent VMs.
- **Version Control for Environments:** Dockerfiles are text files that can be checked into Git, enabling you to track and roll back infrastructure changes.
- **Microservices:** Docker is the foundation of modern microservice architectures where each service runs in its own container.
- **DevOps & CI/CD:** Docker streamlines the pipeline from code commit to production deployment.

1.5 Docker Use Cases

Docker is used across a wide spectrum of scenarios in modern software engineering:

- Microservices architecture — each service in its own container
- CI/CD pipelines — build, test, and deploy in ephemeral containers
- Local development — replicate production environments exactly
- Multi-tenant SaaS platforms — isolate customer workloads
- Machine learning model serving — package model + runtime together
- Database and middleware isolation — run specific DB versions per project
- Edge computing and IoT — deploy consistent containers on edge devices

Chapter 2: Docker Architecture & Core Concepts

To use Docker effectively, it is essential to understand its underlying architecture. Docker uses a client-server architecture. The Docker client talks to the Docker daemon, which does the heavy lifting of building, running, and distributing containers.

2.1 The Docker Daemon (dockerd)

The Docker daemon (dockerd) is a background service that listens for Docker API requests and manages Docker objects such as images, containers, networks, and volumes. The daemon can communicate with other daemons to manage distributed Docker services.

Note: The daemon must be running for any Docker CLI commands to work.

2.2 The Docker Client

The Docker client (docker) is the primary interface through which users interact with Docker. When you type commands like 'docker run' or 'docker build', the client sends these commands to dockerd using the Docker REST API, which carries them out. The client can communicate with more than one daemon.

2.3 Docker Registries

A Docker registry stores Docker images. Docker Hub is the default public registry. When you run 'docker pull ubuntu', Docker looks for the ubuntu image on Docker Hub. Organizations often use private registries such as:

- Docker Hub (hub.docker.com) — the default public registry
- Amazon ECR (Elastic Container Registry)
- Google Artifact Registry
- Azure Container Registry (ACR)
- GitHub Container Registry (ghcr.io)
- Self-hosted registries using the open-source Docker Registry

2.4 Core Docker Objects

Images

A Docker image is a read-only template used to create containers. Images are built from a Dockerfile and consist of layers. Each instruction in a Dockerfile creates a new layer. Layers are cached, which makes rebuilds extremely fast when only a small part of the Dockerfile changes.

Containers

A container is a runnable instance of an image. You can create, start, stop, move, or delete a container using the Docker CLI or API. A container is defined by its image and any configuration options you provide when you create or start it.

Volumes

Volumes are the preferred mechanism for persisting data generated by and used by Docker containers. Volumes are completely managed by Docker and are independent of the container lifecycle — data persists even when the container is deleted.

Networks

Docker networking allows containers to communicate with each other and with the outside world. Docker provides several built-in network drivers: bridge (default), host, overlay, macvlan, and none.

2.5 The Union File System

Docker images use a union file system (UnionFS) to layer changes on top of each other. Popular implementations include OverlayFS (the current default on most Linux systems), AUFS, and devicemapper. The union file system allows multiple read-only layers to be stacked, with a single read-write layer on top representing the running container.

When you modify a file in a running container, a copy-on-write (CoW) mechanism copies the file from the read-only image layer to the read-write container layer before the modification is made. This keeps the underlying image unchanged.

2.6 Linux Kernel Features Powering Docker

Docker relies on several Linux kernel features to provide isolation and resource management:

Feature	Purpose
Namespaces	Process isolation (PID, NET, IPC, MNT, UTS, USER namespaces)
cgroups	Resource limiting (CPU, memory, disk I/O, network)
OverlayFS	Layered, copy-on-write filesystem for images
seccomp	Syscall filtering for security profiles
Capabilities	Fine-grained privilege control (no full root)
AppArmor/SELinux	Mandatory access control profiles

Chapter 3: Installing Docker

Docker is available on Linux, macOS, and Windows. The recommended installation method varies by platform. Docker Desktop is the easiest way to get started on macOS and Windows, while Linux users typically install Docker Engine directly.

3.1 Installing Docker on Ubuntu/Debian

```
# Step 1: Update apt and install prerequisites
sudo apt-get update
sudo apt-get install -y ca-certificates curl gnupg

# Step 2: Add Docker's official GPG key
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
    sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

# Step 3: Set up the repository
echo "deb [arch=$(dpkg --print-architecture) \
    signed-by=/etc/apt/keyrings/docker.gpg] \
    https://download.docker.com/linux/ubuntu \
    $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
    sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Step 4: Install Docker Engine
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io \
    docker-buildx-plugin docker-compose-plugin

# Step 5: Verify installation
sudo docker run hello-world
```

3.2 Post-Installation Steps (Linux)

After installation, you'll typically want to allow your user to run Docker commands without sudo, and configure Docker to start on boot:

```
# Add your user to the docker group
sudo usermod -aG docker $USER

# Apply group changes (or log out and back in)
newgrp docker

# Enable Docker to start on boot
sudo systemctl enable docker.service
sudo systemctl enable containerd.service
```

```
# Start Docker now
sudo systemctl start docker

# Verify you can run without sudo
docker run hello-world
```

3.3 Installing on CentOS/RHEL/Fedora

```
# Install using the convenience script (for testing only)
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh

# Or install via dnf
sudo dnf remove docker docker-client docker-client-latest \
  docker-common docker-latest docker-latest-logrotate \
  docker-logrotate docker-engine
sudo dnf install -y dnf-plugins-core
sudo dnf config-manager --add-repo \
  https://download.docker.com/linux/fedora/docker-ce.repo
sudo dnf install docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
```

3.4 Docker Desktop (macOS & Windows)

Docker Desktop is an application for Mac and Windows machines for building and sharing containerized applications and microservices. It includes Docker Engine, Docker CLI, Docker Compose, Docker Buildx, Docker Scout, and Kubernetes.

- Download from: <https://www.docker.com/products/docker-desktop>
- macOS: Requires macOS 12 or newer. Available for Intel and Apple Silicon (M1/M2/M3).
- Windows: Requires Windows 10/11 64-bit with WSL 2 backend (recommended) or Hyper-V.

3.5 Verifying the Installation

```
# Check Docker version
docker --version
# Output: Docker version 26.x.x, build xxxxxxxx

# Check Docker info (full system info)
docker info

# Run the classic hello-world container
docker run hello-world

# Check Docker Compose version
docker compose version
```


Chapter 4: Working with Docker Images

Docker images are the foundation of containers. Understanding how to find, pull, inspect, tag, and manage images is essential to working with Docker. An image is an immutable, layered package that contains everything needed to run an application.

4.1 Pulling Images

```
# Pull the latest version of an image
docker pull ubuntu

# Pull a specific version/tag
docker pull ubuntu:22.04
docker pull python:3.11-slim
docker pull node:20-alpine

# Pull from a private registry
docker pull myregistry.com:5000/myapp:v1.2.3

# Pull with platform specification (for M1/M2 Macs)
docker pull --platform linux/amd64 ubuntu:22.04
```

4.2 Listing and Inspecting Images

```
# List all local images
docker images
docker image ls

# List with full SHA digests
docker images --digests

# Filter images by name
docker images python

# Show only image IDs
docker images -q

# Inspect image metadata (layers, env vars, entrypoint, etc.)
docker inspect ubuntu:22.04

# Show image history (all layers)
docker history ubuntu:22.04

# Show image size breakdown
docker image ls --format "table {{.Repository}}\t{{.Tag}}\t{{.Size}}"
```

4.3 Tagging Images

Tags are labels you apply to images to help identify them. An image can have multiple tags. Tagging is essential for versioning and organizing your images.

```
# Tag an image
docker tag myapp:latest myapp:v2.1.0
docker tag myapp:latest registry.example.com/team/myapp:v2.1.0

# Tag format: [registry/][namespace/]repository[:tag]
# Examples:
docker tag app:build myuser/app:latest          # Docker Hub
docker tag app:build ghcr.io/myorg/app:v1.0     # GitHub Container Registry
docker tag app:build 123456.dkr.ecr.us-east-1.amazonaws.com/app:latest # ECR
```

4.4 Pushing Images to a Registry

```
# Login to Docker Hub
docker login

# Login to a private registry
docker login registry.example.com

# Login to GitHub Container Registry
echo $GITHUB_TOKEN | docker login ghcr.io -u USERNAME --password-stdin

# Push an image
docker push myuser/myapp:v1.0.0
docker push ghcr.io/myorg/myapp:latest

# Push all tags
docker push --all-tags myuser/myapp
```

4.5 Removing Images

```
# Remove a specific image
docker rmi ubuntu:20.04
docker image rm ubuntu:20.04

# Remove by image ID
docker rmi abc123def456

# Force remove (even if containers use it)
docker rmi -f ubuntu:20.04

# Remove all dangling (untagged) images
docker image prune

# Remove all unused images (not used by any container)
docker image prune -a
```

```
# Remove all images (use with caution!)
docker rmi $(docker images -q)
```

4.6 Saving and Loading Images

You can export and import images as tar archives — useful for air-gapped environments:

```
# Save an image to a tar file
docker save -o myapp.tar myapp:v1.0

# Save multiple images
docker save -o images.tar myapp:v1.0 mydb:5.7

# Load an image from a tar file
docker load -i myapp.tar

# Export a running container's filesystem (not the same as save)
docker export mycontainer -o container.tar

# Import as a new image
docker import container.tar myapp:imported
```

Chapter 5: Dockerfile — Writing & Optimizing

A Dockerfile is a text file containing a series of instructions that Docker reads to automatically build an image. Every line creates a new layer in the image. Writing efficient Dockerfiles is one of the most important skills in Docker mastery.

5.1 Dockerfile Instruction Reference

Instruction	Purpose	Example
FROM	Base image	FROM python:3.11-slim
RUN	Execute command	RUN apt-get update && apt-get install -y curl
COPY	Copy files	COPY . /app
ADD	Copy + extract URLs/tars	ADD app.tar.gz /app
WORKDIR	Set working directory	WORKDIR /app
ENV	Set environment variable	ENV PORT=8080
ARG	Build-time variable	ARG VERSION=1.0
EXPOSE	Document ports	EXPOSE 8080
CMD	Default command	CMD ["python", "app.py"]
ENTRYPOINT	Fixed command	ENTRYPOINT ["gunicorn"]
VOLUME	Create mount point	VOLUME /data
USER	Set user	USER appuser
LABEL	Add metadata	LABEL version=1.0
HEALTHCHECK	Container health check	HEALTHCHECK CMD curl -f http://localhost/
ONBUILD	Trigger on child build	ONBUILD COPY . /app
STOPSIGNAL	Stop signal	STOPSIGNAL SIGTERM
SHELL	Override default shell	SHELL ["/bin/bash", "-c"]

5.2 A Complete Python Application Dockerfile

```
# Use a specific slim base image for smaller size
FROM python:3.11-slim

# Set metadata
LABEL maintainer="team@example.com"
LABEL version="1.0.0"
```

```

LABEL description="FastAPI application"

# Set environment variables
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    APP_HOME=/app

# Install system dependencies
RUN apt-get update && apt-get install -y --no-install-recommends \
    curl \
    && rm -rf /var/lib/apt/lists/*

# Set working directory
WORKDIR $APP_HOME

# Copy and install Python dependencies first (layer caching)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Create a non-root user
RUN addgroup --system appgroup && adduser --system --ingroup appgroup appuser

# Copy application code
COPY --chown=appuser:appgroup . .

# Switch to non-root user
USER appuser

# Document the port the app listens on
EXPOSE 8000

# Health check
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
    CMD curl -f http://localhost:8000/health || exit 1

# Start the application
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

```

5.3 Multi-Stage Builds

Multi-stage builds are one of the most powerful features for optimizing Dockerfiles. They allow you to use multiple FROM statements in a single Dockerfile, using one stage to build your application and a separate, minimal stage to run it. The final image only contains what is needed to run the application, dramatically reducing image size.

```

# Stage 1: Build stage
FROM node:20 AS builder

```

```

WORKDIR /app

# Install dependencies
COPY package*.json ./
RUN npm ci --only=production

# Copy source and build
COPY . .
RUN npm run build

# Stage 2: Production stage (much smaller!)
FROM node:20-alpine AS production

WORKDIR /app

# Copy only what we need from the build stage
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/package.json .

# Run as non-root
USER node

EXPOSE 3000
CMD ["node", "dist/server.js"]

# Result: Image is ~80% smaller than a single-stage build

```

5.4 Dockerfile Best Practices

- **Use specific base image tags** — Avoid 'latest'; pin to exact versions like python:3.11.4-slim for reproducibility.
- **Order instructions by change frequency** — Put frequently changing steps (like COPY . .) near the end to maximize layer cache utilization.
- **Combine RUN commands** — Chain related commands with && to reduce the number of layers.
- **Clean up in the same RUN** — Delete package caches, temp files in the same RUN layer:
'apt-get install ... && rm -rf /var/lib/apt/lists/*'.
- **Use .dockerignore** — Exclude node_modules, .git, __pycache__, *.pyc, and other unnecessary files.
- **Never store secrets in Dockerfiles** — Use build args for non-sensitive values; use runtime secrets or secret mounts for sensitive data.
- **Use COPY instead of ADD** — ADD has extra functionality (auto-extraction) that can be surprising; prefer COPY for clarity.

- **Run as non-root** — Create a dedicated user and switch with USER to follow the principle of least privilege.

5.5 The .dockerignore File

```
# .dockerignore – exclude files from the build context
.git
.gitignore
*.md
node_modules/
__pycache__/
*.pyc
*.pyo
.env
.env.*
dist/
build/
coverage/
.pytest_cache/
.mypy_cache/
.DS_Store
Thumbs.db
Dockerfile
docker-compose*.yaml
tests/
docs/
```

5.6 Build Arguments vs Environment Variables

```
# ARG is only available during build time
ARG APP_VERSION=1.0.0
ARG BUILD_DATE

# ENV persists in the running container
ENV APP_VERSION=$APP_VERSION

# Build with build args
docker build --build-arg APP_VERSION=2.0.0 \
    --build-arg BUILD_DATE=$(date -u +%Y-%m-%dT%H:%M:%SZ) \
    -t myapp:2.0.0 .

# NEVER pass secrets as ARG – they appear in docker history!
# Instead, use Docker secrets or BuildKit secret mounts:
RUN --mount=type=secret,id=mysecret \
    cat /run/secrets/mysecret
```

Chapter 6: Container Lifecycle Management

Containers have a well-defined lifecycle: created, running, paused, stopped, and removed. Understanding how to manage containers at each stage is fundamental to day-to-day Docker usage.

6.1 Running Containers

```
# Basic run
docker run ubuntu echo "Hello from Ubuntu"

# Run interactively with a pseudo-TTY
docker run -it ubuntu /bin/bash

# Run in detached (background) mode
docker run -d nginx

# Run with a name
docker run -d --name my-nginx nginx

# Map ports (host:container)
docker run -d -p 8080:80 nginx

# Set environment variables
docker run -d -e DATABASE_URL=postgres://... myapp

# Mount a volume
docker run -d -v /host/path:/container/path myapp

# Auto-remove on exit
docker run --rm ubuntu ls /

# Limit resources
docker run -d --memory="512m" --cpus="0.5" myapp

# Set restart policy
docker run -d --restart unless-stopped nginx
```

6.2 Managing Running Containers

```
# List running containers
docker ps

# List all containers (including stopped)
docker ps -a

# Show only container IDs
docker ps -q
```



```
# Get detailed info about a container
docker inspect my-nginx

# See container logs
docker logs my-nginx
docker logs -f my-nginx          # Follow (stream) logs
docker logs --tail 100 my-nginx  # Last 100 lines
docker logs --since 1h my-nginx  # Logs from last hour

# Show real-time resource usage
docker stats
docker stats my-nginx

# See running processes inside a container
docker top my-nginx
```

6.3 Interacting with Containers

```
# Execute a command in a running container
docker exec my-nginx ls /etc/nginx

# Open an interactive shell in a running container
docker exec -it my-nginx /bin/bash
docker exec -it my-nginx /bin/sh  # for alpine-based containers

# Copy files to/from containers
docker cp my-nginx:/etc/nginx/nginx.conf ./nginx.conf
docker cp ./nginx.conf my-nginx:/etc/nginx/nginx.conf

# Attach to a running container's I/O streams
docker attach my-nginx  # Use Ctrl+P, Ctrl+Q to detach
```

6.4 Stopping, Starting, and Removing

```
# Stop a container (graceful, sends SIGTERM then SIGKILL after timeout)
docker stop my-nginx
docker stop -t 30 my-nginx  # 30 second timeout

# Kill immediately (SIGKILL)
docker kill my-nginx

# Start a stopped container
docker start my-nginx

# Restart a container
docker restart my-nginx

# Pause / unpause (freezes processes)
docker pause my-nginx
```

```
docker unpause my-nginx
```

```
# Remove a stopped container
```

```
docker rm my-nginx
```

```
# Force remove a running container
```

```
docker rm -f my-nginx
```

```
# Remove all stopped containers
```

```
docker container prune
```

```
# Remove container and its anonymous volumes
```

```
docker rm -v my-nginx
```

6.5 Container Restart Policies

Policy	Description
no	Never restart (default)
on-failure[:N]	Restart only on non-zero exit code, optionally max N times
always	Always restart; starts on Docker daemon start
unless-stopped	Always restart unless explicitly stopped by user

Chapter 7: Docker Networking

Docker networking allows containers to communicate with each other, with the host machine, and with external networks. Docker provides flexible networking options through a pluggable driver architecture. Understanding networking is crucial for multi-container applications.

7.1 Network Drivers Overview

Driver	Use Case	Notes
bridge	Default for standalone containers	Containers on same host can communicate
host	High-performance workloads	Shares host network stack; no isolation
overlay	Multi-host / Swarm networking	Required for Docker Swarm services
macvlan	Legacy apps needing MAC address	Container gets its own MAC on network
ipvlan	More control over IP assignment	L2 and L3 modes available
none	Complete network isolation	Loopback only; no external access

7.2 Managing Networks

```
# List all networks
docker network ls

# Create a custom bridge network
docker network create mynetwork

# Create with custom subnet
docker network create --driver bridge \
  --subnet 192.168.100.0/24 \
  --gateway 192.168.100.1 \
  mynetwork

# Inspect a network
docker network inspect mynetwork

# Connect a container to a network
docker network connect mynetwork my-container

# Disconnect a container from a network
docker network disconnect mynetwork my-container

# Remove a network
docker network rm mynetwork

# Remove all unused networks
```

```
docker network prune
```

7.3 Container-to-Container Communication

Containers on the same user-defined bridge network can communicate using container names as hostnames. This is one of the key reasons to create custom networks rather than using the default bridge.

```
# Create a shared network
docker network create app-network

# Start a database container
docker run -d \
  --name postgres \
  --network app-network \
  -e POSTGRES_PASSWORD=secret \
  postgres:15

# Start app container on same network
# It can reach postgres by hostname 'postgres'
docker run -d \
  --name webapp \
  --network app-network \
  -e DATABASE_URL=postgres://postgres:secret@postgres:5432/mydb \
  -p 8000:8000 \
  myapp:latest
```

7.4 Port Publishing

```
# Publish a specific host port
docker run -p 8080:80 nginx          # host:container

# Publish to all interfaces
docker run -p 0.0.0.0:8080:80 nginx

# Publish to specific interface only
docker run -p 127.0.0.1:8080:80 nginx

# Publish a random host port
docker run -p 80 nginx
docker port container_name          # see the assigned port

# Publish multiple ports
docker run -p 80:80 -p 443:443 nginx

# Publish UDP port
docker run -p 5353:5353/udp dns-server
```

7.5 DNS in Docker

Docker has a built-in DNS server that provides automatic service discovery for containers on user-defined networks. Each container can resolve other containers on the same network by name. The default bridge network does NOT support this feature — only user-defined networks do.

```
# On user-defined networks, containers discover each other by name
# Inside 'webapp' container:
curl http://api-service:3000/endpoint
ping postgres
nslookup redis
```

```
# Custom network aliases allow multiple names
docker run -d \
  --network app-network \
  --network-alias db \
  --network-alias database \
  postgres:15
```

```
# Both 'db' and 'database' resolve to the same container
```

Chapter 8: Volumes & Persistent Storage

By default, data written inside a container is lost when the container is removed. Docker provides three mechanisms for persisting data: volumes (managed by Docker), bind mounts (link to host filesystem), and tmpfs mounts (in-memory only). Volumes are the recommended approach for production use.

8.1 Docker Volumes

```
# Create a named volume
docker volume create mydata

# List volumes
docker volume ls

# Inspect a volume
docker volume inspect mydata

# Mount a volume into a container
docker run -d \
  --name myapp \
  -v mydata:/app/data \
  myimage

# Or using the newer --mount syntax (more explicit)
docker run -d \
  --name myapp \
  --mount type=volume,source=mydata,target=/app/data \
  myimage

# Remove a volume
docker volume rm mydata

# Remove all unused volumes
docker volume prune
```

8.2 Bind Mounts

Bind mounts map a directory or file on the host machine to a path in the container. They are useful for local development where you want code changes to be reflected immediately without rebuilding the image.

```
# Mount current directory into container
docker run -d \
  -v $(pwd):/app \
  -p 3000:3000 \
  node:20 npm run dev
```

```
# Using --mount syntax
docker run -d \
  --mount type=bind,source=$(pwd),target=/app \
  -p 3000:3000 \
  node:20 npm run dev

# Read-only bind mount
docker run -d \
  -v $(pwd)/config:/app/config:ro \
  myapp

# Bind mount a single file
docker run -d \
  -v $(pwd)/nginx.conf:/etc/nginx/nginx.conf:ro \
  nginx
```

8.3 tmpfs Mounts

tmpfs mounts are temporary filesystems stored in the host's memory (and never written to disk). They are useful for sensitive data that should not persist, or for high-performance temporary storage.

```
# Create a tmpfs mount
docker run -d \
  --mount type=tmpfs,destination=/tmp \
  myapp

# With size limit
docker run -d \
  --mount type=tmpfs,destination=/tmp,tmpfs-size=100m \
  myapp

# Legacy syntax
docker run -d \
  --tmpfs /tmp:rw,size=100m \
  myapp
```

8.4 Volume Drivers

Docker supports third-party volume drivers through plugins. These enable volumes backed by cloud storage, distributed file systems, and more:

- local — the default driver, stores on the local host filesystem
- nfs — mount NFS shares as Docker volumes
- azure-file — Azure File Share volumes for Azure deployments
- flocker — multi-host data management platform
- convoy — backup, restore, and migrate volumes

```
# Use NFS volume driver
docker volume create --driver local \
  --opt type=nfs \
  --opt o=addr=192.168.1.100,rw \
  --opt device=:/exports/mydata \
  nfs-volume
```

8.5 Backup & Restore Volumes

```
# Backup a volume to a tar file
docker run --rm \
  -v mydata:/source:ro \
  -v $(pwd):/backup \
  busybox tar czvf /backup/mydata-backup.tar.gz -C /source .
```

```
# Restore a volume from a tar file
docker run --rm \
  -v mydata:/target \
  -v $(pwd):/backup \
  busybox tar xzvf /backup/mydata-backup.tar.gz -C /target
```

```
# Copy data between volumes
docker run --rm \
  -v source-vol:/from:ro \
  -v dest-vol:/to \
  busybox cp -a /from/. /to/
```

8.6 Storage Comparison

Feature	Volumes	Bind Mounts	tmpfs
Managed by Docker	Yes	No	Yes
Host path control	No	Yes	No
Persists after rm	Yes	Yes (host)	No
Portable	Yes	No	No
Best for	Production data	Dev, config files	Sensitive/temp data
Performance	High	High (Linux)	Highest

Chapter 9: Docker Compose

Docker Compose is a tool for defining and running multi-container applications. With Compose, you use a YAML file (docker-compose.yml) to configure all your application's services, networks, and volumes. A single command creates and starts all the services from your configuration.

Starting with Docker Compose V2 (bundled as 'docker compose' instead of 'docker-compose'), Compose is now a first-class plugin integrated directly into the Docker CLI.

9.1 The docker-compose.yml Structure

```
version: '3.9'

services:
  web:
    build: .
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=postgresql://user:pass@db:5432/mydb
      - REDIS_URL=redis://cache:6379
    depends_on:
      db:
        condition: service_healthy
      cache:
        condition: service_started
    volumes:
      - ./app:/app
    restart: unless-stopped
    networks:
      - app-net

  db:
    image: postgres:15
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
      POSTGRES_DB: mydb
    volumes:
      - postgres-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U user -d mydb"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - app-net
```

```
cache:
  image: redis:7-alpine
  volumes:
    - redis-data:/data
  command: redis-server --appendonly yes
  networks:
    - app-net

volumes:
  postgres-data:
  redis-data:

networks:
  app-net:
    driver: bridge
```

9.2 Essential Compose Commands

```
# Start all services in detached mode
docker compose up -d

# Start and rebuild images
docker compose up -d --build

# Stop all services
docker compose down

# Stop and remove volumes
docker compose down -v

# View logs for all services
docker compose logs

# Follow logs for a specific service
docker compose logs -f web

# View running services
docker compose ps

# Scale a service
docker compose up -d --scale web=3

# Run a one-off command in a service container
docker compose run --rm web python manage.py migrate

# Execute in a running container
docker compose exec web bash

# Rebuild a specific service
```

```
docker compose build web
```

```
# Pull latest images
```

```
docker compose pull
```

```
# Restart a specific service
```

```
docker compose restart web
```

9.3 A Full-Stack Application Example

Here's a complete example of a Node.js + PostgreSQL + Redis + Nginx stack:

```
version: '3.9'
```

```
services:
```

```
  nginx:
```

```
    image: nginx:alpine
```

```
    ports:
```

```
      - "80:80"
```

```
      - "443:443"
```

```
    volumes:
```

```
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
```

```
      - ./nginx/ssl:/etc/nginx/ssl:ro
```

```
    depends_on:
```

```
      - api
```

```
    networks:
```

```
      - frontend
```

```
  api:
```

```
    build:
```

```
      context: ./api
```

```
      dockerfile: Dockerfile
```

```
      target: production
```

```
    environment:
```

```
      NODE_ENV: production
```

```
      PORT: 3000
```

```
      DB_HOST: postgres
```

```
      REDIS_HOST: redis
```

```
    env_file:
```

```
      - .env.production
```

```
    depends_on:
```

```
      postgres:
```

```
        condition: service_healthy
```

```
    networks:
```

```
      - frontend
```

```
      - backend
```

```
    restart: unless-stopped
```

```
  worker:
```

```
build:
  context: ./api
  target: production
command: node workers/queue-processor.js
environment:
  NODE_ENV: production
env_file:
  - .env.production
depends_on:
  - postgres
  - redis
networks:
  - backend
restart: unless-stopped
```

```
postgres:
  image: postgres:15-alpine
  environment:
    POSTGRES_DB: ${DB_NAME}
    POSTGRES_USER: ${DB_USER}
    POSTGRES_PASSWORD: ${DB_PASSWORD}
  volumes:
    - pgdata:/var/lib/postgresql/data
    - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U ${DB_USER}"]
    interval: 10s
    timeout: 5s
    retries: 5
  networks:
    - backend
```

```
redis:
  image: redis:7-alpine
  command: >
    redis-server
    --requirepass ${REDIS_PASSWORD}
    --appendonly yes
    --maxmemory 256mb
    --maxmemory-policy allkeys-lru
  volumes:
    - redisdata:/data
  networks:
    - backend
```

```
volumes:
  pgdata:
  redisdata:
```

```
networks:
  frontend:
  backend:
    internal: true # No external access to backend network
```

9.4 Environment Variables in Compose

```
# .env file (automatically loaded by Compose)
DB_NAME=myapp
DB_USER=postgres
DB_PASSWORD=supersecret
REDIS_PASSWORD=redissecret

# Access in docker-compose.yml
environment:
  - POSTGRES_DB=${DB_NAME}
  - POSTGRES_USER=${DB_USER}

# Or with env_file key
env_file:
  - .env
  - .env.local # Overrides for local dev

# Multiple compose files for different environments
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d
```

9.5 Health Checks & Dependencies

```
# Service-level healthcheck
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
  interval: 30s
  timeout: 10s
  retries: 3
  start_period: 40s

# Wait for healthy dependency
depends_on:
  database:
    condition: service_healthy # Wait for healthcheck to pass
  cache:
    condition: service_started # Just wait for container to start
  migration:
    condition: service_completed_successfully # Wait for job to finish
```

9.6 Compose Profiles

Compose profiles allow you to define optional services that are only started when a specific profile is active — great for dev tools, monitoring, or optional features.

```
services:
  api:
    image: myapi
    # No profile = always started

  adminer:
    image: adminer
    profiles: ["dev", "debug"] # Only started with these profiles

  prometheus:
    image: prom/prometheus
    profiles: ["monitoring"]

# Start with profile
docker compose --profile dev up -d
docker compose --profile monitoring up -d

# Start multiple profiles
COMPOSE_PROFILES=dev,monitoring docker compose up -d
```

Chapter 10: Docker Security & Best Practices

Security is a critical concern when running containers in production. Docker provides many built-in security features, and following best practices can significantly reduce your attack surface. Security in Docker is a multi-layered concern spanning image security, runtime security, network security, and secrets management.

10.1 Image Security

- **Use minimal base images** — Alpine Linux (~5MB), distroless, or scratch images minimize the attack surface.
- **Pin image versions** — Never use 'latest' in production. Pin to digest: FROM ubuntu@sha256:abc123...
- **Scan images for vulnerabilities** — Use docker scout, Trivy, Snyk, or Gype to scan before deployment.
- **Verify image signatures** — Docker Content Trust (DCT) ensures only signed images are pulled.
- **Keep images updated** — Regularly rebuild images to pick up OS security patches.

```
# Enable Docker Content Trust
export DOCKER_CONTENT_TRUST=1

# Scan an image with Docker Scout
docker scout cves myapp:latest

# Scan with Trivy (open source)
trivy image myapp:latest

# Check for secrets in images
docker run --rm \
  -v /var/run/docker.sock:/var/run/docker.sock \
  trufflesecurity/trufflehog docker://myapp:latest
```

10.2 Running as Non-Root

```
# In Dockerfile: create and use non-root user
RUN groupadd --gid 1001 appgroup && \
    useradd --uid 1001 --gid appgroup --shell /bin/bash --create-home appuser

USER appuser

# At runtime, verify user
docker run --rm myapp whoami    # Should NOT print 'root'

# Override user at runtime (for debugging only)
docker run --user 1001:1001 myapp
```

```
# Check if container is running as root
docker inspect mycontainer | grep User
```

10.3 Linux Capabilities

By default, Docker drops many Linux capabilities. You can drop all capabilities and add back only what your application needs, following the principle of least privilege.

```
# Drop all capabilities, add only what's needed
docker run --cap-drop ALL --cap-add NET_BIND_SERVICE nginx

# Never use --privileged in production
# docker run --privileged myapp # Gives full host access!

# In docker-compose.yml
cap_drop:
  - ALL
cap_add:
  - NET_BIND_SERVICE
  - CHOWN
```

10.4 Seccomp & AppArmor

```
# Run with custom seccomp profile
docker run --security-opt seccomp=/path/to/profile.json myapp

# Disable seccomp (not recommended for production)
docker run --security-opt seccomp=unconfined myapp

# Run with AppArmor profile
docker run --security-opt apparmor=docker-default myapp

# No new privileges escalation
docker run --security-opt no-new-privileges myapp
```

10.5 Secrets Management

Never store secrets in environment variables (visible in `docker inspect`), Dockerfiles (baked into image layers), or `docker-compose.yml` files committed to version control.

```
# Docker Secrets (Swarm mode)
echo "mysecretpassword" | docker secret create db_password -

# Use in service
docker service create \
  --secret db_password \
  --env DB_PASSWORD_FILE=/run/secrets/db_password \
  myapp
```



```
# In docker-compose.yml
secrets:
  db_password:
    external: true

services:
  api:
    secrets:
      - db_password
    environment:
      DB_PASSWORD_FILE: /run/secrets/db_password

# BuildKit secret mounts (never stored in image layers)
RUN --mount=type=secret,id=npnrc,target=/root/.npmrc \
  npm install
```

10.6 Network Security

- Use internal networks for backend services — set 'internal: true' in Compose to prevent external access.
- Only expose necessary ports — don't bind to 0.0.0.0 unless required; bind to 127.0.0.1 for local-only services.
- Use TLS for inter-service communication in production, especially in Swarm/Kubernetes.
- Implement network policies to control which containers can talk to which.

10.7 Resource Limits

```
# Limit CPU and memory at runtime
docker run -d \
  --memory="512m" \
  --memory-swap="1g" \
  --cpus="0.5" \
  --pids-limit=100 \
  myapp

# In docker-compose.yml
services:
  api:
    deploy:
      resources:
        limits:
          cpus: '0.50'
          memory: 512M
        reservations:
          cpus: '0.25'
          memory: 256M
```

10.8 Security Scanning Workflow

```
#!/bin/bash
# Automated security pipeline

# Build image
docker build -t myapp:${GIT_SHA} .

# Scan for OS vulnerabilities
trivy image --exit-code 1 --severity HIGH,CRITICAL myapp:${GIT_SHA}

# Scan for secrets in the image
trufflehog docker://myapp:${GIT_SHA}

# Check Dockerfile best practices
hadolint Dockerfile

# If all pass, push to registry
docker push myapp:${GIT_SHA}
docker tag myapp:${GIT_SHA} myapp:latest
docker push myapp:latest
```

Chapter 11: Docker in Production

Running Docker in production requires additional considerations around observability, orchestration, CI/CD integration, and operational excellence. This chapter covers patterns and tools used by teams running Docker at scale.

11.1 Logging Best Practices

Containers should log to stdout/stderr, not to files. The Docker daemon captures these streams and routes them through its logging driver system.

```
# Configure logging driver (json-file is default)
docker run -d \
  --log-driver json-file \
  --log-opt max-size=10m \
  --log-opt max-file=3 \
  myapp

# Send logs to a central system
docker run -d \
  --log-driver fluentd \
  --log-opt fluentd-address=localhost:24224 \
  myapp

# Other log drivers: syslog, gelf, awslogs, splunk

# In daemon.json (apply to all containers)
# /etc/docker/daemon.json:
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "10m",
    "max-file": "3"
  }
}
```

11.2 Health Checks in Production

```
# Dockerfile HEALTHCHECK
HEALTHCHECK --interval=30s \
  --timeout=10s \
  --start-period=30s \
  --retries=3 \
  CMD curl -f http://localhost:8000/health || exit 1

# Check health status
docker inspect --format '{{.State.Health.Status}}' myapp
```

```

# Wait for healthy in scripts
while [ "$(docker inspect --format='{{.State.Health.Status}}' myapp)" != "healthy" ]; do
    echo "Waiting for container to be healthy..."
    sleep 2
done
echo "Container is healthy!"

# Health endpoint in your app should return:
# - 200 OK when healthy
# - 503 when degraded
# Include checks for: DB connectivity, cache, disk space

```

11.3 CI/CD Pipeline with Docker

```

# GitHub Actions workflow example
name: Build and Deploy

on:
  push:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Login to GHCR
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${GITHUB_ACTOR}
          password: ${GITHUB_TOKEN}

      - name: Build and push
        uses: docker/build-push-action@v5
        with:
          context: .
          push: true
          tags: |
            ghcr.io/${GITHUB_REPOSITORY}:latest
            ghcr.io/${GITHUB_REPOSITORY}:${GITHUB_SHA}
          cache-from: type=gha
          cache-to: type=gha,mode=max

      - name: Scan image

```

```
uses: aquasecurity/trivy-action@master
with:
  image-ref: ghcr.io/${{ github.repository }}:${{ github.sha }}
  severity: HIGH,CRITICAL
```

11.4 Docker in Kubernetes

While Docker is the most popular way to build container images, Kubernetes (K8s) is the de facto standard for orchestrating them in production. Kubernetes uses container images built by Docker (or Buildkit directly) but manages them with its own runtime (containerd).

- Kubernetes replaces Docker Swarm for most production use cases.
- Images built with docker build work directly in Kubernetes with no changes.
- Docker Desktop includes a local Kubernetes cluster for development.
- Helm charts are the Kubernetes equivalent of Docker Compose files.

```
# Build image for Kubernetes deployment
docker build -t myapp:v1.2.3 .
docker push registry.example.com/myapp:v1.2.3

# Kubernetes Deployment referencing the image
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp
  template:
    spec:
      containers:
      - name: myapp
        image: registry.example.com/myapp:v1.2.3
        resources:
          requests:
            memory: "256Mi"
            cpu: "250m"
          limits:
            memory: "512Mi"
            cpu: "500m"
      livenessProbe:
        httpGet:
          path: /health
          port: 8000
```

```
initialDelaySeconds: 30
```

11.5 Performance Tuning

- **Use BuildKit** — Enable `DOCKER_BUILDKIT=1` for faster builds with parallel execution and better caching.
- **Layer caching in CI** — Use registry-based build caches (`--cache-from`, `--cache-to`) to speed up CI builds.
- **Multi-platform builds** — Use `docker buildx` to build for multiple architectures (AMD64, ARM64) in one command.
- **Minimize image size** — Smaller images pull faster, use less disk, and have smaller attack surfaces.

```
# Enable BuildKit
export DOCKER_BUILDKIT=1

# Or in daemon.json
{ "features": { "buildkit": true } }

# Multi-platform build
docker buildx create --use
docker buildx build \
  --platform linux/amd64,linux/arm64 \
  --push \
  -t myapp:latest .

# Registry cache for CI
docker buildx build \
  --cache-from type=registry,ref=myregistry/myapp:cache \
  --cache-to type=registry,ref=myregistry/myapp:cache,mode=max \
  -t myapp:latest .

# Analyze image size with dive
dive myapp:latest
```

11.6 Quick Reference: Essential Docker Commands

Command	Description
<code>docker ps -a</code>	List all containers
<code>docker images</code>	List all images
<code>docker build -t name:tag .</code>	Build image from Dockerfile
<code>docker run -d -p 8080:80 img</code>	Run container detached

<code>docker exec -it name bash</code>	Shell into running container
<code>docker logs -f name</code>	Stream container logs
<code>docker stop \$(docker ps -q)</code>	Stop all containers
<code>docker system prune -af</code>	Remove all unused resources
<code>docker stats</code>	Live resource usage stats
<code>docker network ls</code>	List all networks
<code>docker volume ls</code>	List all volumes
<code>docker compose up -d</code>	Start all Compose services
<code>docker compose down -v</code>	Stop and remove volumes
<code>docker scout cves img</code>	Scan image for CVEs

Appendix A: Docker Swarm

Docker Swarm is Docker's native clustering and orchestration solution. It turns a pool of Docker hosts into a single, virtual Docker host, enabling high-availability deployments without the complexity of Kubernetes. While Kubernetes has largely replaced Swarm for large-scale production, Swarm is still widely used for simpler setups.

A.1 Initializing a Swarm

```
# Initialize swarm on the first manager node
docker swarm init --advertise-addr 192.168.1.10

# Output includes a join token for workers:
# docker swarm join --token SWMTKN-1-xxx 192.168.1.10:2377

# Get the join token again later
docker swarm join-token worker
docker swarm join-token manager

# Join as a worker node
docker swarm join \
  --token SWMTKN-1-xxx \
  192.168.1.10:2377

# List nodes in the swarm
docker node ls

# Inspect a node
docker node inspect node-name

# Promote a worker to manager
docker node promote worker-node

# Drain a node (for maintenance)
docker node update --availability drain node-name
```

A.2 Deploying Services

```
# Deploy a service
docker service create \
  --name web \
  --replicas 3 \
  --publish published=80,target=80 \
  nginx:alpine

# List services
docker service ls
```



```
# Inspect service details
docker service inspect web

# View service tasks (which node each replica runs on)
docker service ps web

# Scale a service
docker service scale web=5

# Update a service (rolling update)
docker service update \
  --image nginx:1.25 \
  --update-parallelism 1 \
  --update-delay 10s \
  web

# Roll back an update
docker service rollback web

# Remove a service
docker service rm web
```

A.3 Deploying Stacks

A stack is a group of interrelated services that share dependencies and can be scaled and orchestrated together. Stacks use Compose files as their format.

```
# Deploy a stack from a compose file
docker stack deploy -c docker-compose.yml mystack

# List stacks
docker stack ls

# List services in a stack
docker stack services mystack

# List tasks in a stack
docker stack ps mystack

# Remove a stack
docker stack rm mystack

# Update a stack (redploy with changes)
docker stack deploy -c docker-compose.updated.yml mystack
```

A.4 Swarm Secrets

```
# Create a secret from stdin
echo "mysecretpassword" | docker secret create db_password -
```

```

# Create from file
docker secret create ssl_cert ./cert.pem

# List secrets
docker secret ls

# Use in a service
docker service create \
  --name api \
  --secret db_password \
  --env DB_PASSWORD_FILE=/run/secrets/db_password \
  myapi:latest

# In Compose/Stack file:
secrets:
  db_password:
    external: true

services:
  api:
    image: myapi:latest
    secrets:
      - db_password

```

A.5 Swarm Overlay Networks

```

# Create an overlay network for swarm services
docker network create \
  --driver overlay \
  --attachable \
  myoverlay

# Services on this network can communicate across nodes
docker service create \
  --name api \
  --network myoverlay \
  myapi:latest

docker service create \
  --name db \
  --network myoverlay \
  postgres:15

# The 'api' service can reach 'db' by service name
# across any node in the swarm

```

Appendix B: Debugging & Troubleshooting

Debugging containerized applications requires a different mindset than debugging traditional applications. This appendix covers the most common issues and how to diagnose and resolve them efficiently.

B.1 Container Won't Start

```
# Check exit code and logs
docker ps -a                                # See exit code in STATUS column
docker logs <container_id>                 # See stdout/stderr output
docker inspect <container_id>              # Full state, including error

# Common exit codes:
# 0    = Exited normally
# 1    = Application error
# 126  = Permission denied (CMD not executable)
# 127  = Command not found
# 137  = OOM killed (out of memory)
# 139  = Segmentation fault
# 143  = SIGTERM (graceful stop)

# Run an interactive shell to debug the image
docker run -it --entrypoint /bin/sh myapp:latest

# Override the entrypoint to inspect
docker run -it --entrypoint="" myapp:latest /bin/bash
```

B.2 Networking Issues

```
# Check if container can reach another
docker exec myapp ping other-service
docker exec myapp curl http://api:3000/health
docker exec myapp nslookup api

# Inspect network configuration
docker network inspect bridge
docker inspect myapp | grep -A 20 NetworkSettings

# Check what ports are exposed and mapped
docker port myapp
docker inspect myapp | grep -A 10 Ports

# Test DNS resolution inside container
docker exec myapp cat /etc/resolv.conf
docker exec myapp cat /etc/hosts

# Check if port is already in use on host
```

```
sudo ss -tlnp | grep :8080
sudo lsof -i :8080
```

B.3 Volume and Permission Issues

```
# Check volume mounts
docker inspect myapp | grep -A 20 Mounts

# Common issue: files owned by root can't be written by app user
# Fix: set correct ownership in Dockerfile
RUN chown -R appuser:appgroup /app/data

# Or at runtime
docker run --user root myapp chown -R 1001:1001 /app/data

# Check file permissions inside container
docker exec myapp ls -la /app/data

# Volume path debugging
docker volume inspect myvolume
ls -la $(docker volume inspect myvolume | grep Mountpoint | awk -F'"' '{print $4}')
```

B.4 Resource Issues

```
# Real-time resource monitoring
docker stats --no-stream
docker stats --format "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}"

# Check if OOM killed
docker inspect myapp | grep OOMKilled
dmesg | grep -i "oom"

# Check disk usage
docker system df
docker system df -v    # Verbose (per-image breakdown)

# Free up disk space
docker system prune      # Remove stopped containers, unused networks, dangling images
docker system prune -a   # Also remove unused images
docker system prune -a -f --volumes # Include volumes (DESTRUCTIVE!)
```

B.5 Build Failures

```
# Build with no cache (start fresh)
docker build --no-cache -t myapp .

# Show full build output (disable build kit progress)
BUILDKIT_PROGRESS=plain docker build -t myapp .
```

```
# Debug a specific layer – add RUN ls after a failing step
RUN ls -la /app/

# Check what's in the build context
docker build --progress=plain . 2>&1 | grep "sending build context"

# If context is too large, check .dockerignore
echo ".git
node_modules
dist" > .dockerignore

# Inspect intermediate layer (old-style build)
DOCKER_BUILDKIT=0 docker build -t myapp .
docker run -it <intermediate-layer-id> /bin/bash
```

B.6 Useful Diagnostic Commands

Scenario	Command
See all container events	<code>docker events --since 1h</code>
Diff from image (changed files)	<code>docker diff mycontainer</code>
Copy file from container	<code>docker cp mycontainer:/path/file ./local</code>
Run as root for debugging	<code>docker exec -u root -it mycontainer bash</code>
Check container resource limits	<code>docker inspect mycontainer grep -E 'Memory Cpu'</code>
See all open sockets	<code>docker exec mycontainer ss -tulnp</code>
Check DNS	<code>docker exec mycontainer nslookup google.com</code>
Test network connectivity	<code>docker exec mycontainer wget -qO- http://api:3000</code>
Inspect image layers	<code>docker history --no-trunc myimage</code>
See environment variables	<code>docker exec mycontainer env</code>

Appendix C: Running a Private Registry

While public registries like Docker Hub are convenient, many organizations require private registries for security, compliance, or performance reasons. Docker provides an open-source registry image you can self-host.

C.1 Running the Official Registry

```
# Run a basic private registry on port 5000
docker run -d \
  -p 5000:5000 \
  --name registry \
  --restart unless-stopped \
  -v $(pwd)/registry-data:/var/lib/registry \
  registry:2

# Push an image to your private registry
docker tag myapp:latest localhost:5000/myapp:latest
docker push localhost:5000/myapp:latest

# Pull from your private registry
docker pull localhost:5000/myapp:latest

# List images in the registry (Registry API v2)
curl http://localhost:5000/v2/_catalog
curl http://localhost:5000/v2/myapp/tags/list
```

C.2 Registry with Authentication

```
# Create password file using htpasswd
mkdir -p auth
docker run --rm \
  --entrypoint htpasswd \
  httpd:2 -Bbn admin secretpassword > auth/htpasswd

# Run registry with basic auth
docker run -d \
  -p 5000:5000 \
  --name registry \
  -v $(pwd)/auth:/auth \
  -v $(pwd)/registry-data:/var/lib/registry \
  -e REGISTRY_AUTH=htpasswd \
  -e REGISTRY_AUTH_HTPASSWD_REALM="Registry Realm" \
  -e REGISTRY_AUTH_HTPASSWD_PATH=/auth/htpasswd \
  registry:2

# Login to your private registry
```

```
docker login localhost:5000
# Enter username: admin
# Enter password: secretpassword
```

C.3 Registry with TLS

```
# Generate self-signed certificate (for testing)
mkdir -p certs
openssl req -newkey rsa:4096 -nodes -sha256 \
  -keyout certs/domain.key \
  -x509 -days 365 \
  -out certs/domain.crt \
  -subj "/CN=myregistry.example.com"

# Run registry with TLS
docker run -d \
  -p 443:443 \
  --name registry \
  -v $(pwd)/certs:/certs \
  -v $(pwd)/registry-data:/var/lib/registry \
  -e REGISTRY_HTTP_ADDR=0.0.0.0:443 \
  -e REGISTRY_HTTP_TLS_CERTIFICATE=/certs/domain.crt \
  -e REGISTRY_HTTP_TLS_KEY=/certs/domain.key \
  registry:2

# Now push/pull without specifying port:
docker push myregistry.example.com/myapp:latest
```

C.4 Image Lifecycle Management

Managing image tags and versions is critical for a clean, auditable deployment pipeline. A robust tagging strategy ensures you can always identify exactly what is running in production.

```
# Recommended tagging strategy:
# 1. Content-addressable tag (Git SHA)
docker build -t myapp:${GIT_SHA} .

# 2. Semantic version tag
docker tag myapp:${GIT_SHA} myapp:v2.3.1

# 3. Branch tag (for staging)
docker tag myapp:${GIT_SHA} myapp:main

# 4. Latest (points to most recent stable)
docker tag myapp:${GIT_SHA} myapp:latest

# Image pruning policy
# Remove images older than 30 days (using Docker Hub or ECR lifecycle policies)
# Or locally:
```

```
docker image prune -a --filter "until=720h" # 30 days
```

```
# Find and remove large dangling images
```

```
docker images -f dangling=true
```

```
docker image prune
```

C.5 Docker Hub Automated Builds

Docker Hub supports automated builds triggered by GitHub or Bitbucket commits. When you push to a configured branch, Docker Hub automatically builds and publishes the new image. Key configuration options include:

- Source branch → Docker tag mapping (e.g., main → latest, `/^v([0-9.]+)$/` → `{\1}`)
- Build context path — specify a subdirectory if your Dockerfile isn't at the root
- Build arguments — pass ARGs to your Dockerfile from Hub configuration
- Repository links — automatically rebuild when a base image is updated on Hub

Note: For production use, consider GitHub Actions or GitLab CI for more control over build environments, secrets, and scanning.

Appendix D: Glossary of Docker Terms

AppArmor — A Linux security module providing mandatory access control for Docker containers.

Base Image — The starting image in a Dockerfile FROM instruction. Can be 'scratch' for empty images.

Build Cache — Cached intermediate layers from previous builds that speed up subsequent docker build commands.

BuildKit — An improved Docker build engine with parallel builds, cache mounts, and secret mounts.

cgroups — Linux Control Groups — kernel feature Docker uses to limit container resource usage (CPU, memory).

Container — A runnable instance of an image. Isolated process(es) with their own filesystem, network, and PID space.

Container Registry — A service for storing and distributing Docker images. E.g., Docker Hub, ECR, GCR.

Copy-on-Write (CoW) — Storage strategy where image layers are read-only; modifications copy data to the container layer.

Dangling Image — An untagged image, usually a leftover from builds. Prune with docker image prune.

distroless — Google's minimal base images containing only the app and its runtime, no shell or package managers.

Docker Compose — Tool for defining and running multi-container applications via a YAML configuration file.

Docker Content Trust — Feature using Notary to sign and verify Docker image integrity and publisher.

Docker Context — Named configuration for connecting to different Docker hosts or Kubernetes clusters.

Docker Daemon — Background service (dockerd) that manages images, containers, networks, and volumes.

Docker Desktop — GUI application for macOS/Windows that includes Docker Engine, Compose, and Kubernetes.

Docker Hub — Docker's public registry. Default source for official and community images.

Dockerfile — Text file of instructions for building a Docker image, executed top-to-bottom.

ENTRYPOINT — Dockerfile instruction defining the command that always runs; CMD provides default arguments.

Ephemeral — Short-lived. Containers are designed to be ephemeral — stopped and replaced, not patched in place.

Health Check — Command Docker runs periodically to determine if a container is healthy.

Image — Read-only template of layers used to create containers. Built from a Dockerfile.

Image Digest — SHA256 hash uniquely identifying an image's content. Immutable unlike tags.

Layer — A read-only diff created by each Dockerfile instruction. Layers are shared and cached.

Namespace — Linux kernel feature providing isolation for PID, network, IPC, mount, UTS, and user spaces.

Network Bridge — Default Docker network driver. Containers on same bridge can communicate.

Overlay Network — Multi-host Docker network for Swarm services, using VXLAN tunneling.

seccomp — Secure Computing Mode — Linux kernel feature to filter system calls available to containers.

Service — In Docker Swarm, a desired state description for one or more container replicas.

Stack — In Docker Swarm, a group of services defined in a Compose file and deployed together.

Swarm — Docker's native container orchestration and clustering mode.

Tag — Human-readable label applied to an image (e.g., myapp:v1.2.3). Points to a specific image SHA.

tmpfs — Temporary in-memory filesystem mount, not persisted to disk, erased on container stop.

Union File System — Filesystem that overlays multiple directories, presenting them as one (OverlayFS, AUFS).

Volume — Docker-managed persistent storage independent of container lifecycle.

Volume Driver — Plugin enabling volumes backed by external storage systems (NFS, cloud storage, etc.).

Appendix E: Docker Cheat Sheet

Images

<code>docker build -t name:tag .</code>	<code># Build image</code>
<code>docker build --no-cache -t name .</code>	<code># Build without cache</code>
<code>docker pull image:tag</code>	<code># Pull from registry</code>
<code>docker push name:tag</code>	<code># Push to registry</code>
<code>docker images</code>	<code># List images</code>
<code>docker rmi image:tag</code>	<code># Remove image</code>
<code>docker image prune -a</code>	<code># Remove unused images</code>
<code>docker save -o out.tar image</code>	<code># Save to tar</code>
<code>docker load -i out.tar</code>	<code># Load from tar</code>
<code>docker history image</code>	<code># Show layers</code>
<code>docker inspect image</code>	<code># Show metadata</code>

Containers

<code>docker run image</code>	<code># Run container</code>
<code>docker run -d image</code>	<code># Run detached</code>
<code>docker run -it image bash</code>	<code># Interactive shell</code>
<code>docker run --name foo image</code>	<code># Named container</code>
<code>docker run -p 8080:80 image</code>	<code># Port mapping</code>
<code>docker run -e VAR=val image</code>	<code># Environment var</code>
<code>docker run -v /host:/cnt image</code>	<code># Bind mount</code>
<code>docker run --rm image</code>	<code># Auto-remove on exit</code>
<code>docker run --restart=always image</code>	<code># Auto-restart</code>
<code>docker ps</code>	<code># Running containers</code>
<code>docker ps -a</code>	<code># All containers</code>
<code>docker stop foo</code>	<code># Graceful stop</code>
<code>docker kill foo</code>	<code># Force stop</code>
<code>docker start foo</code>	<code># Start stopped</code>
<code>docker restart foo</code>	<code># Restart</code>
<code>docker rm foo</code>	<code># Remove stopped</code>
<code>docker rm -f foo</code>	<code># Force remove</code>
<code>docker exec -it foo bash</code>	<code># Shell in container</code>
<code>docker logs -f foo</code>	<code># Stream logs</code>
<code>docker stats</code>	<code># Resource usage</code>
<code>docker inspect foo</code>	<code># Full metadata</code>
<code>docker cp foo:/path ./local</code>	<code># Copy file out</code>

Networks

<code>docker network ls</code>	<code># List networks</code>
<code>docker network create mynet</code>	<code># Create bridge</code>
<code>docker network inspect mynet</code>	<code># Inspect network</code>
<code>docker network connect mynet foo</code>	<code># Connect container</code>

```
docker network disconnect mynet foo # Disconnect
docker network rm mynet             # Remove network
docker network prune                 # Remove unused
```

Volumes

```
docker volume create myvol          # Create volume
docker volume ls                     # List volumes
docker volume inspect myvol         # Inspect volume
docker volume rm myvol              # Remove volume
docker volume prune                  # Remove unused
```

Docker Compose

```
docker compose up -d                # Start detached
docker compose up --build            # Rebuild and start
docker compose down                  # Stop and remove
docker compose down -v               # Also remove volumes
docker compose ps                    # Status
docker compose logs -f               # Stream all logs
docker compose logs -f service       # Specific service
docker compose exec service bash     # Shell in service
docker compose run --rm svc cmd      # One-off command
docker compose pull                  # Pull latest images
docker compose build                  # Build images
docker compose restart service       # Restart service
docker compose scale web=3           # Scale service
```

System Maintenance

```
docker system df                     # Disk usage
docker system df -v                  # Verbose breakdown
docker system prune                  # Remove unused resources
docker system prune -a               # Include unused images
docker system prune -a -f --volumes # Everything (careful!)
docker system info                    # System-wide info
docker system events                  # Live event stream
docker version                        # Client/server versions
```

Swarm

```
docker swarm init                    # Initialize swarm
docker swarm join --token T host     # Join as worker
docker node ls                       # List nodes
docker service create --name s img   # Create service
docker service ls                     # List services
docker service ps service            # List tasks
docker service scale svc=5           # Scale service
docker service update --image img s # Rolling update
```

<code>docker service rollback s</code>	<code># Roll back</code>
<code>docker stack deploy -c file.yml s</code>	<code># Deploy stack</code>
<code>docker stack ls</code>	<code># List stacks</code>
<code>docker stack rm s</code>	<code># Remove stack</code>
<code>docker secret create name file</code>	<code># Create secret</code>
<code>docker secret ls</code>	<code># List secrets</code>